

I Left **Port 22** Open for **147 Days** Disguised as a Solana Validator.

Here's What Crawled In.

136,128

SESSIONS

6,145

UNIQUE IPS

414K

COMMANDS

147

DAYS

PLATFORM	Microsoft Azure VM • Ubuntu 22.04 LTS
PERIOD	April 27 – September 21, 2026
ENGINE	Custom Python SSH Honeypot • asyncssh
PERSONA	Solana Validator Node v2.1.16
AUTHOR	Jenna • Hot Pink Huntress

TABLE OF CONTENTS

PRE	Pre-Investigation Notes	
	Operator sessions excluded. Term definitions.	
01	Executive Summary	
	The experiment, the scale, the takeaway.	
02	Methodology	
	What the decoy was. How it was built.	
03	Key Findings at a Glance	
	136,128 sessions. 414,070 commands. 147 days.	
04	Attacker Infrastructure	
	Countries, ASNs, cloud vs residential, the outlier.	
05	What They Did When They Got In	
	Commands, behavior tiers, credentials, return visitors.	
06	When They Were Active	
	Hourly distribution. The corrected peak times.	
07	MITRE ATT&CK Mapping	
	17 techniques. Discovery dominates.	
08	Notable Sessions	
	French backdoor. Miner squad. First knock. Five wallet hunters.	
09	Hands on the Keyboard	
	Two sessions that show someone was actually there.	
10	Indicators of Compromise	
	11 defanged IPs. Verify before blocking.	
11	What You Should Do About It	
	7 actionable takeaways.	
12	Limitations	
	Honeypot caveats. Data gaps. Honest accounting.	

// TL;DR – FIVE THINGS TO KNOW

- **The lure didn't matter.** 147 days as a "Solana validator" and 136,128 sessions later — only 5 found the fake wallet. Attackers treated this box as generic Linux compute.
- **Automation dominates.** 96.6% of sessions ran zero or one command. The internet runs on scripts, not humans.
- **The main goal was crypto mining, not Solana keys.** 3,692 sessions deployed a miner. 5 looked for wallet files.
- **They came back.** 95.5% of sessions came from IPs that had connected before. One successful brute-force doesn't generate one intrusion — it generates a subscription.
- **Turn off SSH password auth.** Every session in this dataset succeeded because the honeypot accepted every credential. Public-key-only authentication would have stopped all of it.

// Pre-Investigation Notes

Five addresses were excluded from all metrics in this report: four operator-controlled addresses (including loopback) and one friend who knew about the project and tested it.

//

SECTION 1

Executive Summary

For 147 calendar days, a server sat on the open internet with SSH port 22 exposed, pretending to be a Solana cryptocurrency validator node. It accepted every login attempt automatically, logged every command, and said nothing. No real data was at risk. The point was to watch: who shows up, what do they want, and how do they behave once they think they're in?

Over that period, 6,145 unique IP addresses knocked on the door. They generated 136,128 sessions, tried 63,283 distinct username-and-password combinations, and issued 414,070 commands after gaining access. The server was hit with an average of 926 sessions every single day. The moment the honeypot went live, the first attacker arrived. Not hours later. The moment it went live.

The attackers were not, for the most part, sophisticated human hackers typing commands one by one. The vast majority of activity came from automated tools running credential wordlists and deployment scripts. The goal in most cases was cryptocurrency mining: get in, kill any competing miner already running, drop a new miner binary, set up persistence via cron and systemd, and leave. A smaller subset of sessions existed to validate whether a credential worked, with no exploitation attempted at all. At the far end of the spectrum, a handful of sessions showed genuine human operators conducting methodical reconnaissance and attempting to plant persistent backdoors.

The single most actionable takeaway from this dataset: **disable password-based SSH authentication**. Nearly every session in this report was possible because the server accepted a password login (560 attempted public-key auth). Switch to SSH key authentication, and the attack surface for all of this activity drops to near zero.

//

SECTION 2

Methodology

The decoy. The server presented itself as a Solana blockchain validator node. The SSH banner, fake filesystem layout, and simulated process list all reinforced this fiction. User accounts named `sol` and `solana` appeared in the environment. The goal was to make the server look like it was running real Solana validator software, to see whether attackers would behave differently toward a known crypto-infrastructure target.

Tech stack. The honeypot ran on an Azure virtual machine. A custom Python SSH server (built on the `asyncssh` library) accepted all incoming connections, simulated a Linux shell, and logged every credential attempt and every command to a SQLite database. The shell responses were scripted to look plausible without executing anything real.

Run period. April 27, 2026 through September 21, 2026. 147 calendar days.

Credential acceptance. Every username and password combination was accepted. Attackers were never told their credentials were wrong. This is standard honeypot practice: rejection teaches the attacker nothing useful happened, while acceptance draws them deeper and reveals behavior.

Term definitions.

- *Session:* A single SSH connection from authentication through disconnect.
- *Connection:* Used interchangeably with session throughout this report.
- *Return visitor:* An IP address that created more than one session over the run period.
- *High-interest session:* A session that triggered one or more behavioral flags indicating substantive attacker activity beyond a simple credential probe.
- *Easter egg:* A named behavioral trap embedded in the simulated environment. Specific commands or actions trigger these flags, which are used to classify attacker intent and depth.

Operator exclusions. As noted above, five addresses are excluded from all metrics.

Playtester. One visitor on day 2 found the easter egg — a playable terminal game launched by typing `top` — and died six times in four minutes. Their sessions have been excluded from all metrics in this report. They knew about the project.

AbuseIPDB limitation. This report does not include AbuseIPDB reputation data. The honeypot database was not populated with AbuseIPDB lookups during the run period, so abuse scores are unavailable for cross-referencing.

//

SECTION 3

Key Findings at a Glance

METRIC	VALUE
Run period	April 27 to September 21, 2026 (147 calendar days)
Total sessions	136,128
Unique attacker IPs	6,145
Unique credential combos tried	63,283
Sessions with commands	69,436 (51.0%)
Total commands logged	414,070
Return-visitor sessions	129,956 (95.5% of total)
Unique IPs that returned	3,830 of 6,145
High-interest sessions	4,994
Daily average	926 sessions/day

136,128

TOTAL SESSIONS

6,145

UNIQUE IPS

414,070

COMMANDS LOGGED

63,283

CREDENTIAL COMBOS

95.5%

RETURN-VISITOR RATE

147

DAYS ACTIVE

The honeypot recorded its first session at 23:09:48 UTC on April 27, 2026. The IP was `2[.]57[.]122[.]238`, geolocated to Amsterdam, Netherlands, operating on AS47890 (Unmanaged LTD). The attacker used username `sol` and password `12345678`. Within seconds of gaining access, the session issued four commands:

```
/bin/./uname -s -v -n -r -m  
uptime -p  
lspci | grep VGA  
nvidia-smi -q
```

The first two are standard reconnaissance. The last two are GPU enumeration. `lspci | grep VGA` lists graphics cards. `nvidia-smi -q` queries NVIDIA GPU status and driver information. This attacker was not looking for Solana tooling. The attacker was hunting for GPU compute capacity to mine cryptocurrency. The server had been live for less than a minute.

The 95.5% return-visitor rate deserves special note. Only 4.5% of sessions in this dataset came from an IP that had never connected before. The other 95.5% came from IPs that had connected at least once previously — 129,956 sessions (95.5%). This is not loyalty. It is scripted re-scanning. Automated tools log working credentials and return to them. 3,830 of the 6,145 unique attacker IPs returned at least once. The implication for defenders is that a successful brute-force attempt does not generate one intrusion event. It generates dozens or hundreds.

//

SECTION 4

Attacker Infrastructure

4a. Where They Came From

RANK	COUNTRY	SESSIONS	UNIQUE IPS
1	Germany	32,535	166
2	Pakistan	22,289	217
3	United Kingdom	22,223	63
4	Mexico	9,327	31
5	China	6,617	1,086
6	India	6,148	574
7	United States	5,338	596
8	Estonia	4,122	7
9	Hong Kong	2,730	321
10	Turkey	2,040	18

Two patterns stand out. Germany's 32,535 sessions came from 166 IP addresses. That is a session-to-IP ratio of 196:1. The United Kingdom's 22,223 sessions came from 63 IPs, a ratio of 353:1. Both of these reflect concentrated scanning infrastructure: a small number of machines generating enormous traffic volumes. China, by contrast, produced 6,617 sessions from 1,086 unique IPs, a ratio of 6:1. The Chinese traffic is more distributed, consistent with a larger pool of compromised or rented devices rather than a handful of dedicated scanners.

One important note on geography: country attribution reflects where an IP address is registered or geolocated, not the nationality of the person operating the tool. Attackers route traffic through VPNs, rented servers, and compromised machines in other countries. Germany being number one in session volume does not mean German attackers. It means German-hosted infrastructure.

4b. Cloud vs. Residential

CATEGORY	SESSIONS	UNIQUE IPS	SHARE OF IPS
Cloud (VPS/datacenter)	80,340	1,473	59.0%
Residential/commercial	55,803	4,672	41.0%

A corrected classification using ASN-based rules shows 80,340 sessions (59.0%) originating from hosting providers, VPS platforms, and CDN/transit networks, with 55,803 sessions (41.0%) from ISP and residential blocks. The original `is_cloud` field in the database undercounts hosting by a factor of three — it missed large providers like Play2go International (29,314 sessions) and Cogent Communications (8,951 sessions) which were labeled residential. The takeaway from the earlier report that attacks came predominantly from "compromised residential devices" does not hold; the majority of traffic originated from rented cloud and VPS infrastructure.

4c. Top ASNs

RANK	ASN	NAME	SESSIONS	IPS
1	AS215439	Play2go International Limited	29,314	13
2	AS23966	LINKdotNET Telecom Limited	21,087	2
3	AS16276	OVH SAS	14,075	57
4	AS174	Cogent Communications	8,951	3
5	AS47890	Unmanaged LTD	7,218	4
6	AS8075	Microsoft Corporation	6,398	553
7	AS41745	Baykov Ilya Sergeevich	4,085	4
8	AS4837	China Unicom	1,255	122
9	AS4766	Korea Telecom	1,208	401
10	AS24961	WIIT AG	1,166	1

AS215439 (Play2go International Limited) delivered 29,314 sessions from 13 IP addresses. That is 2,255 sessions per IP. AS23966 (LINKdotNET Telecom Limited, Pakistan) produced 21,087 sessions from 2 IPs, averaging 10,544 sessions per IP. These are not distributed botnets. These are purpose-built scanning nodes running at high velocity from static addresses.

AS8075 is Microsoft Corporation, the backbone of Azure. 6,398 sessions from 553 IPs means Azure-hosted machines were scanning this honeypot. Cloud platforms prohibit outbound port scanning in their terms of service. The presence of 553 distinct Microsoft-hosted IPs in this dataset suggests a meaningful number of compromised or policy-violating Azure instances are participating in SSH brute-force campaigns.

AS41745 (Baykov Ilya Sergeevich) registered under an individual's name produced 4,085 sessions from 4 IPs. Individual-registration ASNs operating at this volume are a consistent pattern in threat infrastructure.

4d. The Credential-Validation Wave

On May 22, 2026, between 02:46 and 10:56 UTC, IP address `38[.]58[.]169[.]236` (AS174 Cogent Communications, geolocated to Mexico) created 8,949 sessions over 8 hours and 10 minutes. Every single session followed an identical pattern: one command, the credential-validation ping:

```
echo -e "\x6F\x6B"
```

This outputs the string `ok`. Nothing else. No reconnaissance. No payload. No persistence.

This is not an attack. It is a quality-assurance sweep. The operator had a list of credentials and wanted to confirm which ones still worked. Each session connected, sent the ping, confirmed the shell responded, and disconnected. The result is a verified list of live credentials — which can feed exploitation tooling, or be sold on underground markets where buyers pay for confirmed working logins rather than raw wordlists.

This wave accounts for 8,949 of the 63,364 total `echo -e "\x6F\x6B"` commands in the dataset. It is also why the previous report's analysis showed an artifact peak in the 02:00 to 10:00 UTC window. Once this single IP is excluded, the hourly distribution shifts (detailed in Section 6).

What They Did When They Got In

5a. Top Commands

RANK	COMMAND	COUNT	WHAT IT MEANS
1	<code>echo -e "\x6F\x6B"</code>	63,364	Credential-validation ping. Confirms the shell is live and responding. Outputs "ok."
2	<code>kill -9 \$(lsof +D ...)</code>	62,453	Competitor-process killer. Finds processes with open file handles in a target directory and kills them. Used to evict other miners.
3	<code>readlink -f /proc/root/exe</code>	45,475	Honeygot artifact: the miner script reads "root" from the fake <code>ps aux</code> output and passes it to <code>readlink -f /proc/root/exe</code> . On real Linux, <code>/proc/</code> directories are numbered by PID, not username — this path does not exist on real systems. The fake shell responded as if it were valid.
4	<code>kill -9 root</code>	45,191	Honeygot artifact: the script passes the username "root" (read from fake <code>ps aux</code>) where a numeric PID is required. On a real Linux system, <code>kill -9 root</code> would fail. On actual compromised servers, this script correctly resolves numeric PIDs from <code>ps</code> output. Part of the miner competitor-eviction loop.
5	<code>readlink -f /proc/solana/exe</code>	17,171	Honeygot artifact: same as the <code>/proc/root/exe</code> case, but for the "solana" username from fake <code>ps aux</code> . Path does not exist on real Linux; the fake shell responded as valid. Part of the eviction loop.
6	<code>kill -9 solana</code>	17,068	Honeygot artifact: passes the username "solana" where a PID is expected. On real Linux this would fail. This is NOT targeted Solana blockchain tooling — the script kills by the process owner's username, and "solana" happened to be a user account on this server.

RANK	COMMAND	COUNT	WHAT IT MEANS
7	<code>cd /dev/shm \ \ </code> <code>... && cat ></code> <code>astats</code>	10,218	Navigates to an in-memory filesystem (invisible to disk-based monitoring) and writes a binary named <code>astats</code> . This is the miner payload drop. The <code>cat > astats</code> write operation appears in the same command chain; both forms represent the same campaign action.
8	<code>uname -a</code>	4,778	Standard system fingerprinting. Returns kernel version, architecture, and hostname.
9	<code>pwd</code>	4,106	Confirms current working directory. Orientation check after login.

Additional commands of note: `chmod +x w.sh` appeared 3,637 times (making a deployed shell script executable), crontab manipulation appeared 3,605 times (establishing scheduled persistence), and systemd unit writes appeared 3,590 times (creating a service that restarts the miner if it gets killed).

The GPU-hunting commands from the first session (`lspci | grep VGA` and `nvidia-smi -q`) appear in a smaller subset of sessions. They signal attacker awareness that cryptocurrency mining benefits from GPU compute. The GPU-hunting commands are not specific to Solana — validators on the Solana network do not require GPU hardware. The attacker that connected first was not confused about what they were looking for.

5b. Three Tiers of Attacker Behavior

The 136,128 sessions split into four clean, non-overlapping tiers by command count. All four sum to the total.

Tier 1: Silent authenticators — 66,692 sessions (49.0%). These sessions connect, authenticate, run zero commands, and disconnect. The shell never received a single keystroke. These are pure credential probers: the tool confirms the password works and logs the result for later. Nothing was deployed; nothing was enumerated. Nearly half the dataset is this.

Tier 2: Single-command validators — 64,339 sessions (47.3%). One command, then gone. The canonical example is the 38[,]58[,]169[,]236 wave (8,949 sessions of `echo -e "\x6F\x6B"`) — a liveness check that confirms the shell is interactive. Other single-command sessions run `uname -a` or `id`. The goal is credential validation, not exploitation. Tiers 1 and 2 together account for 131,031 sessions — 96.3% of all activity.

Tier 3: Active recon and escalation — 1,073 sessions (0.8%). Two to twenty commands: directory exploration, process enumeration, network configuration review, credential harvesting from `/etc/passwd` and `/etc/shadow`. Some sessions attempt to establish persistent access through SSH key injection or account creation. Small in count, high in intent. A subset of these sessions show timing and behavioral signals consistent with human operators — see the Hands on the Keyboard section.

Tier 4: Deployment campaigns — 4,024 sessions (3.0%). More than 20 commands. This tier is dominated by the 93-command miner script (3,210 sessions): it deploys a cryptocurrency miner binary, sets cron and systemd persistence, and runs a kill-competitor loop. Within Tier 4 sits the French backdoor session — 44 commands in the first 80 seconds (connection remained open for 10 hours) — and the 383 sessions that wrote SSH authorized keys. Easter egg triggers provide a behavioral signal within this tier: 4,124 distinct sessions (spanning Tiers 3 and 4) triggered at least one trap. Among specific traps, 3,693 sessions triggered `run_payload`, 3,643 triggered `chmod_exe`, 311 triggered `passwd_harvest`, and 5 reached the `crown_jewel` wallet trap.

5c. Credentials They Tried

Password patterns:

PATTERN	COUNT
Custom (leaked or wordlist)	50,562
Dictionary word	30,905
Numeric only	29,313
IoT default	18,344
Service default	4,872
Keyboard walk	1,455
Public key	559
Crypto-related	103

Top 10 passwords:

RANK	PASSWORD	COUNT
1	admin123	15,938
2	support	2,219
3	admin	1,615
4	1234	586
5	123456	387
6	root	386
7	(blank)	290
8	password	266
9	ubuntu	235
10	ubnt	221

Top 10 usernames:

RANK	USERNAME	COUNT
1	root	68,468
2	azureuser	24,911
3	admin	21,967
4	support	2,628
5	user	1,648
6	ubuntu	986
7	testuser	780
8	ubnt	709
9	test	619
10	debian	614

`azureuser` ranking second among targeted usernames is a notable signal. This is not a generic default. `azureuser` is the default administrative account created by Azure when provisioning a Linux virtual machine. Its presence at rank two in the credential list indicates that attacker wordlists have been updated to reflect Azure's default naming convention. Operators running Azure VMs who kept the default `azureuser` account with password authentication enabled are a specific, identified target.

The password `g5=$TkXxp=a6oM!` appeared 119 times. A randomized-looking credential tried 119 times across this dataset suggests it originated from a leaked credentials list, not a brute-force generator. Someone had this specific password associated with a real account somewhere, and the credential ended up in a wordlist that multiple scanning tools are running.

290 sessions attempted a blank password. 559 sessions attempted public key authentication. Both are instrumentation data points: the blank attempts probe for misconfigured accounts, while the public key attempts suggest attackers who have already obtained a private key from a prior compromise and are testing it across new targets.

The `crypto_related` category (103 matches) carries a caveat. The classifier identifies passwords containing terms associated with cryptocurrency, but it also picks up partial matches. The word "elizabeth" would match, for example, if the classifier keys on substring patterns. The 103 number

is an overcount of crypto-themed credentials.

5d. Return Visitors

129,956 sessions (95.5% of the total) came from IP addresses that had connected before. 3,830 of the 6,145 unique attacker IPs in the dataset returned at least once. (See Section 4d for full detail.)

Top 5 IPs by session count:

IP	COUNTRY	ASN	SESSIONS
89[.]39[.]121[.]98	Germany	AS215439 Play2go	29,230
210[.]2[.]144[.]34	Pakistan	AS23966 LINKdotNET	21,075
51[.]89[.]235[.]201	United Kingdom	AS16276 OVH	13,646
38[.]58[.]169[.]236	Mexico	AS174 Cogent	8,949
138[.]124[.]4[.]81	Estonia	AS41745 Baykov	4,081

These five IPs alone account for 76,981 sessions, 56.5% of the dataset total. The return behavior is not curiosity. It is automation. These tools connect, authenticate, execute their script, disconnect, and loop. Every successful credential accepted during the run period put the server on a list that these tools would revisit on a schedule. For defenders, this means that a brute-force success event is not a one-time intrusion. It is enrollment in a recurring campaign.

//

SECTION 6

When They Were Active

Hourly session distribution (operator IPs and 38[.]58[.]169[.]236 excluded):

UTC HOUR	SESSIONS	UTC HOUR	SESSIONS
00:00	3,843	12:00	6,804
01:00	3,704	13:00	6,935
02:00	3,744	14:00	8,994
03:00	3,692	15:00	7,750
04:00	3,819	16:00	6,390
05:00	4,708	17:00	5,213
06:00	4,716	18:00	5,468
07:00	4,323	19:00	8,408
08:00	4,784	20:00	5,093
09:00	5,029	21:00	4,077
10:00	5,829	22:00	3,697
11:00	6,405	23:00	3,739

Two clear peaks emerge once the 38[.]58[.]169[.]236 credential-validation wave is excluded from the data. The largest is 14:00 UTC with 8,994 sessions. The second is 19:00 UTC with 8,408 sessions. The overnight baseline runs between 3,692 and 3,843 sessions per hour from midnight through 04:00 UTC.

14:00 UTC corresponds to 10:00 AM EDT, 4:00 PM CEST, 7:00 PM PKT/IST. The afternoon peak is consistent with business-hours activity across multiple active source regions simultaneously. 19:00 UTC corresponds to 3:00 PM EDT, 9:00 PM CEST. The evening secondary peak is consistent with a

shift in active scanning infrastructure toward US-centric or Americas-centric operators.

A prior version of this report noted an anomalous 02:00 to 10:00 UTC peak. That pattern was an artifact. `38[.]58[.]169[.]236` ran its entire credential-validation sweep between 02:46 and 10:56 UTC on May 22, 2026, producing a concentrated burst that skewed the hourly distribution. With that IP excluded, the true shape of the data is a midday build, an afternoon peak, a moderate evening secondary peak, and a quiet overnight floor.

//

SECTION 7

MITRE ATT&CK Mapping

Counts are sessions per technique; one session can map to multiple techniques; rows do not sum to the total. "% of sessions" is out of 136,128.

TECHNIQUE	ID	TACTIC	SESSIONS	% OF SESSIONS
Brute Force	T1110	Credential Access	136,128	100%
Valid Accounts	T1078	Defense Evasion / Persistence	136,128	100%
System Information Discovery	T1082	Discovery	5,644	4.1%
File and Directory Permissions Modification: Linux	T1222.002	Defense Evasion	4,281	3.1%
Process Discovery	T1057	Discovery	4,175	3.1%
Resource Hijacking (crypto mining)	T1496	Impact	3,692	2.7%
Hide Artifacts: Hidden Files and Directories	T1564.001	Defense Evasion	3,862	2.8%
Command and Scripting Interpreter: Unix Shell	T1059.004	Execution	69,436	51.0%
Scheduled Task/Job: Cron	T1053.003	Persistence	3,654	2.7%
Create or Modify System Process: Systemd Service	T1543.002	Persistence	3,590	2.6%
OS Credential Dumping: /etc/passwd and /etc/shadow	T1003.008	Credential Access	933	0.7%
System Owner/User Discovery	T1033	Discovery	803	0.6%
Ingress Tool Transfer	T1105	Command and Control	782	0.6%
File and Directory Discovery	T1083	Discovery	569	0.4%
System Network Configuration Discovery	T1016	Discovery	509	0.4%

TECHNIQUE	ID	TACTIC	SESSIONS	% OF SESSIONS
Virtualization/Sandbox Evasion: System Checks (honeypot fingerprinting)	T1497.001	Defense Evasion	382	0.3%
System Network Connections Discovery	T1049	Discovery	383	0.3%
Account Manipulation: SSH Authorized Keys	T1098.004	Persistence	383	0.3%
Account Discovery: Local Account (/etc/passwd only)	T1087.001	Discovery	132	0.1%
Indicator Removal: Clear Command History	T1070.003	Defense Evasion	34	0.025%
Impair Defenses: Impair Command History Logging	T1562.003	Defense Evasion	34	0.025%
Abuse Elevation Control Mechanism: Sudo	T1548.003	Privilege Escalation	29	0.021%
Create Account: Local Account	T1136.001	Persistence	25	0.018%
Command and Scripting Interpreter: Python	T1059.006	Execution	12	0.009%

Note on T1497.001 and T1049 (same campaign): The original database tagged 382 sessions with T1552.007 (Container API Credentials). On examination, these sessions run `test -f /.dockerenv`, `test -f /run/.containerenv`, and `ls /opt/cowrie /home/richard /etc/cowrie` — they are fingerprinting for Docker containers and Cowrie honeypot installations, not accessing real container API credentials. The behavior is a fit for T1497.001 (Virtualization/Sandbox Evasion: System Checks). The same 376 of these 382 sessions are also the source of the T1049 (System Network Connections Discovery) count: the Cowrie-detection campaign runs `netstat -tulpn` as part of its recon template, which drives that tag. T1049 without this campaign = 7 sessions. T1098.004 (SSH key writing, 383 sessions) is a *completely separate* campaign with zero session overlap with T1497.001. See the sidebar in the Notable Sessions section.

Note on T1059.004 count: The value 69,436 is all command-bearing sessions — every session that issued at least one command (Tiers 2+3+4: 64,339 + 1,073 + 4,024). SSH exec-channel commands invoke the user's shell the same way interactive sessions do (sshd passes the command string to the shell), so every session that ran commands qualifies for T1059.004. The database's internal `mitre_tags` column tagged only 3,654 sessions (the PTY/interactive subset); that tagging was incomplete. T1053.003 (Cron, 3,654) is a subset of the miner-campaign sessions and is no longer a tie with T1059.004.

Discovery techniques dominate the top of the table, which reflects the structure of the miner deployment campaign: recon first, then execution. The persistence techniques (T1053.003 Cron, T1098.004 SSH Keys, T1136.001 Create Account) map to the miner campaign's cron/systemd setup and the French backdoor session's hidden user creation. The credential-dumping technique (T1003.008 at 933 sessions) corresponds to the `passwd_harvest` easter egg triggers and represents sessions where attackers attempted to read `/etc/passwd` or `/etc/shadow`. The defense-evasion techniques at the bottom (T1070.003 Clear Command History, 34 sessions) mark the subset of attackers who knew to clean up after themselves.

//

SECTION 8

Notable Sessions

8a. The French Backdoor Factory

Session ID `2b0e14c1` arrived on June 18, 2026 at 11:09 UTC from IP `212[.]112[.]19[.]166`, registered to Glesys AB on AS42708 in Finland. The session ran 44 commands over its lifetime, making it one of the most command-dense sessions in the dataset.

The first phase was standard reconnaissance. The attacker checked system information, verified the current user context, explored the directory structure, and confirmed network configuration. Nothing unusual. Then the attacker pasted a multi-part shell payload written in French.

The payload was structured with labeled sections. `DÉTECTION OS` ran conditional checks to identify whether the system was running FreeBSD, a Debian-based distribution, or a Red Hat-based distribution. Each branch executed the correct commands for that OS family. `CRÉATION DES UTILISATEURS` created two hidden user accounts. The first was `sys_admin` with its home directory set to `/var/.sys_admin`. The second was `r00t` with its home directory at `/var/.sys_temp`. Both accounts received the password `Hackers666`. Both were added to the `sudo` group on Debian-based systems and the `wheel` group on Red Hat-based systems, giving them full administrative privileges. `DOSSIERS + PERMISSIONS` created the hidden home directories and set appropriate ownership. `NETTOYAGE SILENCIEUX` (silent cleanup) disabled bash history for both accounts by setting `HISTFILE=/dev/null` and `HISTSIZE=0`, ensuring no commands run under those accounts would be logged.

The entire payload was constructed to produce no stdout output. Every command was redirected to `/dev/null` or suppressed. An observer watching terminal output would see nothing happen.

This session scored 8 on the dataset's internal severity scale, the highest assigned score. The execution was automated — all 44 commands fired in under 80 seconds from a standard recon template — but the payload itself was clearly hand-crafted: full French-language inline comments, multi-distribution OS detection logic, and proper error handling. Most commodity attack tooling is written in English and pulled from shared repositories. This script reads like something built by a specific operator for their own use. The attacker demonstrated awareness of stealth techniques (hidden directories in `/var`, suppressed output, history deletion) and privilege escalation paths across multiple Linux distributions. The automation means this operator was not typing in real time, but someone did the engineering work.

8b. The Miner Eviction Squad

3,210 sessions across multiple source IPs ran an identical 93-command script. The script's internal logic is consistent enough that it almost certainly originated from a shared source: a GitHub repository, an underground forum post, or a direct toolkit transfer between operators.

The script executes in a fixed sequence. First, `uname` and related commands fingerprint the system. Second, the script searches for a writable directory, with `/dev/shm` as the preferred target (it is an in-memory filesystem, meaning the dropped files do not touch disk and are invisible to disk-based monitoring tools). Third, the script writes `w.sh` inline using `cat` with a heredoc, then runs `chmod +x w.sh` to make it executable. Fourth, the script establishes two persistence mechanisms: a crontab entry and a systemd unit file named `watcher-netai.service`. The systemd unit watches for the miner process and restarts it if killed. Fifth, the script runs a kill-competitor loop, repeating 12 or more times: `readlink -f /proc/root/exe` followed by `kill -9 root`, then `readlink -f /proc/solana/exe` followed by `kill -9 solana`. Finally, the script drops a binary named `astats` into `/dev/shm`.

The `kill -9 solana` command requires context. The miner script does not know the Solana service PID directly. Instead, it reads usernames from the fake `ps aux` output — "root" and "solana" — and passes them to both `readlink -f /proc/<username>/exe` and `kill -9 <username>`. On real Linux, `/proc/` directories are numbered by PID, so `/proc/solana/exe` does not exist and `kill -9 solana` (which expects a numeric PID) would fail. These commands are a honeypot artifact: the custom fake shell responded to `/proc/root/exe` and `/proc/solana/exe` as if they were valid paths, producing logged output that doesn't reflect what would happen on a real target. On actual compromised servers, this script correctly resolves numeric PIDs from `ps` output. The kill loop is generic competitor-eviction — any miner running as a user named "solana" would be targeted, regardless of whether Solana-the-blockchain is involved.

The `astats` binary name is an indicator of compromise. Any server with an `astats` binary in `/dev/shm` or another temp-filesystem location should treat it as a strong signal of this campaign.

8c. The First Knock

At 23:09:48 UTC on April 27, 2026, the honeypot recorded its first session. The server had been live for less than a minute. The IP was `2[.]57[.]122[.]238`, geolocated to Amsterdam, Netherlands, on AS47890 (Unmanaged LTD). The attacker used username `sol` and password `12345678`.

The four commands in the session tell a clear story:

```
/bin/./uname -s -v -n -r -m  
uptime -p  
lspci | grep VGA  
nvidia-smi -q
```

The first command uses an unusual path (`/bin/./uname`) that may reflect an attempt to evade command-logging tools that filter on exact command strings. The output contains kernel version, hostname, and architecture. The second command checks system uptime. The third queries PCI hardware for VGA/GPU devices. The fourth runs NVIDIA's system management interface in full query mode, returning detailed information about GPU drivers, memory, temperature, and utilization.

This attacker was GPU hunting. The server's Solana validator fiction may have drawn this specific session, as the attacker used username `sol`. However, Solana validators do not require GPU hardware — this session was hunting for cryptocurrency mining capacity generally. The speed of this first contact, under 60 seconds from server launch, reflects the density of automated scanning on the public internet. A new server on port 22 does not get hours to settle in. It gets seconds.

The ASN AS47890 (Unmanaged LTD) appears throughout this dataset. Three high-volume IPs from AS47890 are among the top IOC entries in Section 10 (2[.]57[.]121[.]25, 2[.]57[.]121[.]112, and 193[.]46[.]255[.]86), along with 2[.]57[.]122[.]238 — the very first attacker, which contributed a single session. Together this ASN produced 7,218 total sessions. The first session was the first of many from this network.

The Five Who Found the Wallet

Five sessions triggered the `crown_jewel` easter egg — meaning they navigated to and read `wallet.json`, the fake Solana hot wallet in the simulated filesystem. These are the only on-thesis evidence in the dataset that the Solana fiction attracted meaningful attacker attention.

SESSION	IP (DEFANGED)	ASN	DATE	HOW THEY GOT THERE
f6385669	144[.]172[.]99[.]18	AS14956 RouterHosting LLC (US)	Jun 3, 2026	Manual step-by-step navigation: <code>cd ..</code> , <code>ls</code> , <code>cd home</code> , typo <code>cx ..</code> , misspelled <code>solona</code> , Windows <code>dir</code> command eventually reached <code>/home/solana</code> and ran <code>cat private_keys_backup.t</code> then <code>cat wallet.json</code> commands, 243 seconds. Confidence: likely human.
a164ca21	173[.]66[.]243[.]238	AS701 Verizon Business (US)	Jun 16, 2026	Direct: <code>cd</code> , <code>ls</code> , <code>cat wallet.json</code> . 3 commands, 142 seconds. This was the first of 21 sessions from this IP that day — see Hands on the Keyboard section.
0536f088	37[.]153[.]130[.]70	AS58224 Iran Telecommunication Company (IR)	Jul 10, 2026	Ran <code>ls</code> , <code>top</code> (played the in-shell game briefly), <code>htc</code> then <code>cat wallet.json</code> commands, 110 seconds.
a79e32c5	18[.]171[.]210[.]244	AS16509 Amazon.com Inc. (UK)	Jul 12, 2026	Ran <code>ls</code> twice, then <code>cat validator-keypair.js</code> then <code>cat wallet.json</code> commands, 45 seconds.
ddedacd4	18[.]171[.]210[.]244	AS16509 Amazon.com Inc. (UK)	Jul 12, 2026	Same IP, 8 minutes later. Ran <code>ls</code> once, session ended. <code>cat</code> command, 14 seconds. Likely the same actor returning to confirm access or grab additional files.

Two sessions came from the same Amazon IP (18[.]171[.]210[.]244), 8 minutes apart on July 12. The first grabbed both the validator keypair and the wallet; the second returned and ran a single `ls`. Whether that's confirmation of access, a second tool in a pipeline, or the same operator

checking their work is inference — but the timing is consistent with one person revisiting. Together, the five sessions come from four distinct IPs across three continents — North America (RouterHosting US and Verizon US), Asia (Iran TCI), and Europe (Amazon UK) — with no behavioral overlap suggesting a coordinated actor.

None of the five navigated using `find` or `grep` on wallet filenames — all either browsed directory-by-directory or already knew the path. Three of the five used `ls` to orient before opening any file. The RouterHosting session (`f6385669`) found the wallet in 4 minutes of manual exploration; the Verizon session (`a164ca21`) got there in 3 commands.

//

SECTION 9

Hands on the Keyboard

Most of what happens in this dataset is automation on top of automation: scanners finding open ports, tools validating credentials, scripts running the same 93 commands in sequence. But a small number of sessions have timing, behavior, and mistakes that point to a human being at the other end. Here are the ones worth naming.

Methodology: sessions were scored on five signals — inter-command timing variance, shell escape attempts, typos and corrections, reactive navigation (commands that reference output from prior responses), and behavioral uniqueness. Timing was computed from the events table timestamps. All sessions below were reviewed command-by-command. Confidence labels are: "likely human or AI-driven agent" where timing is consistent but automation cannot be ruled out; "likely human" where multiple non-timing signals converge. IPs on residential and business ISPs are withheld; only ISP and region are shown.

The Explorer — Washington DC metro area, Verizon FiOS

Sessions: 21 visits on June 16, 2026, plus two later returns. **ISP:** Verizon Business FiOS, Washington DC metro area. **Confidence:** likely human or AI-driven agent.

The same source connected 21 times in six hours, always as `labuser` with passwords that progressively shortened: `password123` → `wer` → `nn` → `n` → `f` → `d` → `g` → (empty) → `a`. Someone was iterating on logins, learning what the server would accept. On the first visit, three commands in: `cat wallet.json`. Crown jewel. Then they came back.

Subsequent sessions show systematic exploration. They read `README.txt`, checked `.bash_history` and `.ssh/authorized_keys`, examined cron entries and systemd units, and spent three separate sessions on `DO_NOT_OPEN.zip` — first using Python's `chr()` mapping to extract printable characters, then reading raw binary, then running a hex-encoded `import os; os.system('ls -la /')` to try to break out of the shell through Python.

They also probed the shell environment directly: `echo $PROMPT_COMMAND`, `unset PROMPT_COMMAND`, `echo $TMOUT`, `export TMOUT=0`. Then tried `/bin/bash --norc --noprofile` to spawn a different shell. The session stayed 5 minutes past each attempt with no success, then reconnected and tried something else.

Timing: Across 105 inter-command gaps in all sessions — minimum 0.0 s (multi-line commands logged separately), median 12.7 s, mean 29.2 s, max 175.6 s, stdev 38.3 s. The distribution is consistent with a person reading output and deciding what to try next, though a sufficiently agentic system could also produce this pattern.

Easter eggs triggered: `crown_jewel`, `self_destruct` (twice), `dare_zip` (three times), `python_repl` (six times).

The Navigator — RouterHosting LLC VPS (US)

Session: f6385669, June 3, 2026. **ASN:** AS14956 RouterHosting LLC (US). **Confidence:** likely human.

Found the wallet in 4 minutes through navigating the fake filesystem with visible uncertainty. The command sequence:

```
cd ..      → ls      → cd home    → ls
cx ..      ← typo    → solana     ← misspelled 'solana'
ls         → cd solana ← still wrong → ls
cat solana → cat solana → s          ← abbreviated
ls         → cd ubuntu → s          → ls
cd .       → cd ..    → s          → ls
cd solana  → ls
cat private_keys_backup.txt
[... 1m 37s pause ...]
cat wallet.json
```

Two typos (`cx ..` for `cd ..`, `solana` for `solana`), command abbreviations (`s`, presumably meaning `ls`), and a 97-second gap before the final `cat wallet.json` after reading the fake private keys. The session reads like someone carefully going through a filesystem they've never seen before, not a tool executing a path it was given.

Sidebar: The Bots That Checked for Honey pots

Not human — but worth a mention. A campaign of 382 sessions (from multiple source IPs, including 128[.]24[.]160[.]229) ran an automated recon script that explicitly tested for Cowrie, the most commonly deployed SSH honeypot:

```
ls /opt/cowrie /home/richard /etc/cowrie 2>&1 # Cowrie default paths (/home/richard is
test -f /.dockerenv                            # Docker container check
test -f /run/.containerenv                     # Podman container check
```

This is a Cowrie-aware recon framework. The honeypot is not Cowrie — it's a custom Python implementation — so every check returned empty. The framework got a false negative and proceeded with its payload. A Cowrie deployment presumably would have been fingerprinted as a honeypot and skipped. These sessions drove the T1497.001 row in the ATT&CK table, and their `netstat -tulpn` calls account for most of the T1049 (System Network Connections Discovery) count.

//

SECTION 10

Indicators of Compromise

The following IPs were the most active or most notable in the dataset. All addresses are defanged — all dots replaced with [.] following threat intelligence community convention to prevent accidental hyperlinking or automated scanning.

IP (DEFANGED)	ASN	COUNTRY	SESSIONS	NOTES
89[.]39[.]121[.]98	AS215439 Play2go International	Germany	29,230	High-volume scanner
210[.]2[.]144[.]34	AS23966 LINKdotNET Telecom	Pakistan	21,075	High-volume scanner
51[.]89[.]235[.]201	AS16276 OVH SAS	United Kingdom	13,646	High-volume scanner
38[.]58[.]169[.]236	AS174 Cogent Communications	Mexico	8,949	Credential-validation sweep only
138[.]124[.]4[.]81	AS41745 Baykov Ilya Sergeevich	Estonia	4,081	High-volume scanner
2[.]57[.]121[.]25	AS47890 Unmanaged LTD	United Kingdom	2,676	Scanner
2[.]57[.]121[.]112	AS47890 Unmanaged LTD	United Kingdom	2,451	Scanner
193[.]46[.]255[.]86	AS47890 Unmanaged LTD	United Kingdom	2,089	Scanner
212[.]112[.]19[.]166	AS42708 Glesys AB	Finland	score-8	French backdoor payload, hidden user creation
50[.]56[.]159[.]185	AS33070 Rackspace	United States	score-7	wget, privilege escalation, file deletion
35[.]243[.]242[.]201	AS396982 Google LLC	United States	score-7	wget, curl, run_payload

Before blocking or reporting any of these IPs, verify their current status. IP addresses are reassigned. A cloud IP that hosted a scanner in May 2026 may belong to a legitimate business in October 2026. Use this list as a starting point for investigation, not an automatic blocklist.

//

SECTION 11

What You Should Do About It

Password authentication on SSH is the root cause of everything in this report. Disable it. Open your SSH server configuration at `/etc/ssh/sshd_config` and set `PasswordAuthentication no`. Generate an SSH key pair with `ssh-keygen -t ed25519`, add the public key to `~/.ssh/authorized_keys` on your server, and confirm you can log in with the key before restarting the SSH service. Once key authentication works, no brute-force wordlist in the world can get in without your private key.

Moving SSH off port 22 reduces noise, not risk. Port 22 is the default. Move SSH to a high-numbered port (above 1024, something non-obvious), and automated scanners targeting port 22 will stop knocking. This is not a security control. A determined attacker will port-scan you. But it reduces log volume and eliminates low-effort commodity scanning from most of your authentication logs, which makes genuine threats easier to see.

If you are running Azure, audit your `azureuser` account immediately. `azureuser` was the second-most targeted username in this dataset, behind only `root`. Azure creates this account by default on Linux VMs. If your VM still has `azureuser` with password authentication enabled, your login credentials are being tried by automated tools right now. Rename the account, disable password auth, and restrict SSH access to known source IPs using Azure Network Security Groups if possible.

The Solana lure mostly didn't change attacker behavior. The honeypot was disguised as a Solana validator, but the majority of attackers treated it as a generic Linux server — credential stuffing, miner deployment, and reconnaissance scripts that are identical on any target. The first attacker used username `sol` and ran GPU enumeration commands (GPU hunting is relevant to crypto mining, not specifically to Solana validators, which do not require GPUs). Only 5 sessions out of 136,128 reached the `wallet.json` trap. The `crypto_related` password category (103 sessions) is an upper bound — the classifier catches partial substrings and may overcount. The miner-eviction loop killed processes running under the `solana` system user, but that is a generic account name in Linux Solana installs, not Solana-specific tooling. What the data supports: SSH-exposed servers are targeted by automated miner-deployment campaigns regardless of what they claim to be, and Solana infrastructure operators face the same baseline SSH risk as any internet-facing Linux server.

Most traffic came from rented hosting infrastructure, not residential botnets. After correcting the ASN classification (see methodology), 59.0% of sessions (80,340) originated from hosting providers, VPS platforms, and CDN/transit networks. The 41.0% residential and ISP traffic is substantial but not dominant. This suggests that most automated scanning and credential-validation campaigns are being run from rented cloud infrastructure — cheap, fast, and disposable — rather than compromised home devices.

A rapid-fire series of successful authentications with no subsequent activity is an active-attack signal, not a normal login pattern. Standard intrusion detection focuses on failed authentication (the "too many wrong passwords" pattern). The 38[.]58[.]169[.]236 event in this dataset produced 8,949 successful authentications in 8 hours with zero exploitation. Traditional fail-ban tools would not have flagged it. Configure monitoring to alert on any IP that authenticates successfully more than a threshold number of times (five is reasonable) within a short window, especially if session depth (command count) stays at zero or one.

After any suspected compromise, check cron and systemd before you check anything else. The miner deployment campaign in this dataset established persistence through two mechanisms: a crontab entry and a systemd unit file named `watcher-netai.service`. Run `crontab -l` for each user account, check `/etc/cron.d/`, review `systemctl list-units --type=service` for unfamiliar service names, and inspect `/etc/systemd/system/` for unit files you did not create. Also check `/dev/shm` for binaries: any executable file in an in-memory filesystem location is suspicious by default. A file named `astats` in `/dev/shm` is a specific indicator for the campaign documented in this report.

//

SECTION 12

Limitations

The honeypot is detectable. Sophisticated attackers running fingerprinting tools can identify a simulated SSH shell. A real Linux system responds differently than a scripted honeypot in edge cases. Attackers who detected the honeypot would have either left or changed their behavior. The dataset captures behavior from attackers who did not detect the simulation, or who continued anyway. The true distribution of attacker sophistication in the wild may differ from what is represented here.

AbuseIPDB was not populated during the run. This report does not include reputation scores from AbuseIPDB or similar threat intelligence platforms. Cross-referencing the top IPs against known-malicious registries would strengthen the IOC section but was not part of the data collection process.

The `crypto_related` password classifier overcounts. The 103 passwords flagged as crypto-related include partial string matches. Any password containing substrings that happen to overlap with cryptocurrency terms (coin names, wallet-related words, or common token tickers) would be captured by this classifier. The 103 figure should be treated as an upper bound, not a precise count of passwords crafted with cryptocurrency systems in mind.

Geolocation carries uncertainty. IP-to-country attribution uses the registered location of the IP block, not the physical location of the device or operator. VPNs, proxy servers, and Tor exit nodes can place an attack's apparent origin in a country that has nothing to do with the actual operator. Country-level statistics in this report describe where traffic came from, not where attackers are.

Comparison to prior reports requires caution. This report is the definitive record of the full 147-day run. Interim reports produced during the run period used different exclusion sets, different run-length denominators, and in some cases included the 38[.]58[.]169[.]236 sessions in hourly distribution data. Numbers from prior reports should not be compared to numbers in this report without accounting for these differences.

The shell simulation affects command data. The honeypot responded to commands with scripted output, not real system responses. Attackers may have issued different follow-on commands than they would have on a real system based on the simulated responses they received. Command counts and sequences reflect behavior in a simulated environment.

The miner campaign skews command-count statistics. The 93-command miner deployment script ran in 3,210 sessions, contributing a concentrated volume of specific commands. Any command that appears in that script (readlink, kill, chmod, crontab operations, systemd writes) is skewed upward in the overall command frequency table. The top-10 command list is partially a portrait of that single campaign rather than a representative cross-section of all attacker activity.

// Closing

147 days of data collected while doing absolutely nothing except existing on the internet with port 22 open. No provocation. No interaction. Just presence. The internet sent 136,128 sessions, 414,070 commands, and one very motivated French-speaking backdoor architect who probably thought they were having a great Thursday.

The lesson is the same one it has always been, and it will still be the same lesson in 147 more days: password authentication on SSH is a door that millions of automated tools know how to knock on, and they will never stop knocking

Jenna | Hot Pink Huntress